Heaps, Stacks and Queues

Topic 1 of 10

38 questions | skeleton and writing space only

Each question: the prompt, the function or class signature, and ruled lines to write your own solution.

1. Median of Two Sorted Arrays (LC 4) with a Follow-up Variant

HARD | asked 2026-10-09 | binary-search, heap, divide-and-conquer

L5 R1 coding: LC 4 (Median of Two Sorted Arrays).

```
def median_two(a, b):
```

2. Compute Turnstile Crossing Times

HARD | asked 2026-05-04 |

Overview: This question evaluates proficiency in queue simulation, event-driven scheduling, and handling priority and tie-breaking rules while achieving linear time complexity.

```
def turnstile_times(events, idle_first='exit'):
```

3. Find minimum threshold enabling grid path

HARD | asked 2026-02-11 |

Overview: This question evaluates a candidate's understanding of grid pathfinding, reachability under constraints, and algorithmic optimization, focusing on skills such as graph traversal and handling threshold-based feasibility.

```
def min_max_path(H):
```

4. Compute minimax grid path and network delay

HARD | asked 2026-01-22 |

Overview: This question evaluates algorithmic problem-solving in shortest-path and path-optimization contexts, specifically minimax pathfinding on a grid and single-source reachability/timing in a weighted directed graph, testing graph modeling, alternative path cost metrics, edge-case handling, and working with large numeric weights.

```
def minimax_path(H):
```

5. Return top-k subset sums in descending order

HARD | asked 2026-01-07 |

Overview: This question evaluates proficiency in combinatorial reasoning, algorithmic optimization for large search spaces, and efficient extraction of top-k values within the coding & algorithms domain.

```
def top_k_sums(nums, k):
```

6. Minimize travel time with optional meeting point

HARD | asked 2026-01-06 |

Overview: This question evaluates a candidate's understanding of graph shortest-path computation and meeting-point optimization, testing skills in handling weighted edges, aggregating path costs, and reasoning about simultaneous movements on a network.

```
def meeting_point(n, edges, a, b, dest, directe
  d=False):
```

7. Minimize maximum height along a grid path

HARD | asked 2026-01-04 |

Overview: This question evaluates algorithmic pathfinding and graph-modeling skills, specifically reasoning about grid-based route selection where the objective is defined by an aggregate maximum over visited cell heights.

```
def min_max_path(H, with_path=False):
```

8. Compute city skyline outline

HARD | asked 2025-12-08 |

You are given a list of rectangular buildings in a 2D city skyline. Each building is represented by three integers [L, R, H]: - L: the x-coordinate of the left edge of the building, - R: the x-coordinate of the right edge of the building (L < R), - H: the height of the building. All buildings stand on a flat horizontal line (y = 0) and extend vertically up to height H. Buildings may overlap in the x-dimension.

```
def skyline(buildings):
```

9. Merge k sorted linked lists.

HARD | asked n/a |

Merge K Sorted Linked Lists. You are given an array of k linked-lists lists, each linked-list is sorted in ascending order. Your task is to merge all the linked-lists into one sorted linked-list and return it. Examples: If lists = [[1,4,5],[1,3,4],[2,6]], where each array represents a linked-list, the merged and sorted linked-list would be [1,1,2,3,4,4,5,6]. The output for the merged list should be in array form for simplicity.

```
def merge_k_heap(lists):
```

10. Visible People in a Line (Last Person, Then Everyone)

MEDIUM | asked 2026-10-07 | array, monotonic-stack, stack

Given heights of people in a line, A can see B if nobody between them is taller than both. Part 1: count how many people the last person can see. Part 2: return that count for every person. Written in a plain editor with no compilation. Input: an array of heights representing a line of people. Visibility rule: person A can see person B if no one standing between them is taller than both A and B. Part 1: return how many people the last person in line can see.

```
def last_sees(a):
```

11. Meeting Rooms / Interval Scheduling Variants

MEDIUM | asked 2026-09-29 | interval, heap, greedy

LC 253 (Meeting Rooms II) appears in many forms across phone screen and onsite coding rounds -- minimum number of rooms, room assignment, cyclic time, daily server jobs, gate entry/exit. The base task in every variant: given a list of intervals (start, end) -- or pickup/return pairs, meeting time pairs, server job runs -- answer one or more of: Min rooms / cars / servers needed to honor all intervals concurrently (the canonical min-meeting-rooms problem).

```
def min_rooms(intervals):
```

12. Active Communication Top Users

MEDIUM | asked 2026-09-22 | hashmap, set, sorting, top-k, streaming

Process a continuous stream of user pairs such as A-B and B-C; each pair creates an active communication edge, and the system should output users ordered by current communication count. An additional API variant exposes register and mostActiveUser(), then asks for constant-time most-active lookup and a changing-K ranking extension.

```
class Active:

    def __init__(self, count_self=False):
```

```
    def bump(self, user):
```

```
    def add(self, a, b):
```

```
    def most_active(self):
```

```
    def top_k(self, k):
```

13. Sliding-Window Mean After Removing the Largest K Values

MEDIUM | asked 2026-09-01 |

For every contiguous window of a fixed size, remove the largest k values by multiplicity and compute the mean of the remaining values. Process sliding updates incrementally rather than sorting each complete window again. Implement trimmed_window_means(nums: int[], window: int, k: int) -> string[]. This exact fraction encoding is an explicit practice representation that avoids cross-language floating-point rounding differences.

```
def trimmed_window_means(nums, window, k, both=
  False):
```

14. Implement Python's itertools.tee

MEDIUM | asked 2026-08-20 | iterator, queue, data-structure, object-design

Implement the behavior of Python's tee: accept one iterator and an integer n, then return n independent iterators over the same source sequence. Define the required function and classes yourself and improve a naive design toward an efficient shared-state implementation.

```
def tee_queues(iterable, n=2):
```

15. Union of Sorted Interval Lists

MEDIUM | asked 2026-08-16 | interval, two-pointer, heap, streaming

Reverse of LC 986: given two individually sorted, internally non-overlapping interval lists, return their union.

```
def union_two(a, b, touching=True):
```

16. Expand Recursive Key-Value Placeholders in a Template

MEDIUM | asked 2026-08-04 |

Overview: Expand %KEY% placeholders in a template, recursively resolving placeholders inside mapped values. Use memoized dependency traversal to preserve literal text, avoid repeated work, and explain cycle, unknown-key, and escaping policies.

```
def expand(template, mapping, unknown='keep', l
  imit=10 ** 6):
```

17. Streaming Median (LC 295) with Follow-ups

MEDIUM | asked 2026-08-03 | heap, streaming

A near-canonical LC 295 streaming-median implementation using two heaps, followed by two design extensions: exploit a very small value domain, then support an arbitrary k-th-smallest query instead of only the median.

```
class MedianFinder:

    def __init__(self):
```

```
    def add(self, x):
```

```
    def median(self):
```

18. Decode String

MEDIUM | asked 2026-07-29 | string, stack, parsing

Examples: "3[a]2[bc]" -> "aaabcbc". "2[abc]3[cd]ef" -> "abcabccdcdcdef". "3[a2[c]]" -> "accaccacc".

```
def decode(s):
```

19. Batch and Streaming Log Deduplication

MEDIUM | asked 2026-07-23 | dedup, streaming, heap, sorting

Process (timestamp, message) logs by deduplicating on message and sorting retained entries by timestamp, first keeping the first-seen entry and then the latest timestamp. The streaming extension adds add(timestamp, message) and get_next(), requiring a map plus min-heap, lazy deletion, and a discussion of watermarks and bounded state.

```
def dedupe(logs, policy='first'):
```

20. Top-K Frequent in Large Logs

MEDIUM | asked 2026-07-19 | heap, hashmap, scaling

Examples: Input: chatLog = ["A says hi", "B says hi", "A says bye", "C says hi", "A says yo"], k=2 Counts: A=3, B=1, C=1 Output: [A, B] // or [A, C] -- ties allowed

```
def top_k_speakers(chat_log, k):
```

21. Find A Threshold-Limited Path With Minimum Required Safety

MEDIUM | asked 2026-07-06 |

Overview: Practice a graph path interview problem that asks for the minimum required safety value along a threshold-limited path. The prompt targets minimax path reasoning, filtered edges, Dijkstra-style relaxation, disconnected graphs, and edge-case handling.

```
def best_path_safety(n, edges, s, t, threshold=
  None):
```

22. Nested Function Expression Evaluator

MEDIUM | asked 2026-06-10 | parsing, recursion, stack

Examples: add(1, sub(1, 0)) -> 2 add(2, mul(3, pow(4, 5))) -> 3074

```
def evaluate(expr, env=None):
```

23. Solve Rooms and Top-K Streams

MEDIUM | asked 2026-06-05 |

Overview: This question evaluates algorithmic problem-solving skills across interval scheduling/resource allocation and streaming top-k maintenance, testing competency in data structures for ordered queries, tie-breaking logic, stateful stream processing, and analysis of time and space trade-offs.

```
def min_rooms(intervals):
```

24. Rearrange Tasks With Cooldown

MEDIUM | asked 2026-06-03 |

Overview: This question evaluates task-scheduling and constraint-handling skills, testing algorithmic reasoning about arranging repeated jobs with cooldown constraints along with analysis of schedule optimality and algorithmic complexity.

```
def least_interval(tasks, n):
```

25. Determine if a 14-tile hand is winning

MEDIUM | asked 2026-03-09 |

You are given an integer array tiles of length 14 representing a Mahjong-like hand. Each integer is a tile value from 1 to 9 (single suit). You may reorder the tiles. A hand is **winning** if it can be partitioned into: - **1 pair**: two identical tiles, and - **4 melds** (each meld uses 3 tiles), where each meld is either: - a **triplet**: three identical tiles (e.g., 7,7,7), or - a **sequence**: three consecutive values (e.g., 2,3,4). Each tile can be used at most once.

```
def is_winning_hand(tiles):
```

26. Implement frequency + distance top-K queries

MEDIUM | asked 2026-02-12 |

You will implement solutions for two coding interview questions. ## Question 1: Return the top-K points by frequency (with distance tie-break) You are given an array points of 2D integer points, where each point is represented as [x, y]. The array may contain duplicates (the same [x, y] can appear multiple times). Define: - freq(p) = number of times point p appears in the array. - dist(p) = squared Euclidean distance to the origin, x^2 + y^2.

```
def top_points(points, k):
```

27. Compute minimum number of rooms needed

MEDIUM | asked 2026-02-12 |

Problem You are given a list of meetings, each with a start time and end time. A single room can host only one meeting at a time. Two meetings **overlap if one starts before the other ends (treat meetings as half-open intervals **[start, end)** so a meeting ending at time t does not conflict with another starting at time t). ### Task Return the **minimum number of rooms** required to host all meetings.**

```
def min_meeting_rooms(intervals):
```

28. Find longest increasing contiguous subarray

MEDIUM | asked 2026-01-06 |

Overview: This question evaluates proficiency in array processing and algorithmic optimization, focusing on recognition of contiguous subarray properties and handling a single-value modification to achieve strict monotonicity.

```
def longest_one_change(a, integers=True):
```

29. Simulate room assignments for scheduled meetings

MEDIUM | asked 2026-01-06 |

Overview: This question evaluates scheduling and event-simulation skills, including time-ordered processing, resource allocation with tie-breaking, and efficient tracking of room availability and usage counts using appropriate data structures.

```
def most_booked_room(n, meetings, by='count'):
```

30. Compute servers needed for daily recurring jobs

MEDIUM | asked 2025-11-05 |

You operate a cluster of identical servers that run recurring **daily jobs. For a single day, you are given a list of job execution intervals. Each interval has: - job_id (string or integer), - start_time (timestamp within that day), - end_time (timestamp within that day), with start_time < end_time. Interpret each interval as a half-open time range [start_time, end_time) during which that job must be running on some server **on that day**.**

```
def servers_needed(intervals, dedicated=()):
```

31. Compute shortest paths with blocked nodes

MEDIUM | asked 2025-09-06 |

Given a graph with nodes and edges and a designated source node s, compute the shortest distance from s to every other node. Some nodes are inaccessible (blocked) and cannot be traversed or reached; treat them as removed from the graph. Return a distance for each node where unreachable nodes (including blocked ones) are -1. Describe your algorithm, data structures, and time/space complexity, and note any differences for directed vs. undirected graphs.

```
def blocked_distances(n, edges, source, blocked
  , directed=False):
```

32. Find the shortest path in a graph.

MEDIUM | asked n/a |

Find the shortest path in a graph.

```
def dijkstra_path(edges, s, t):
```

33. Given a train schedule with departure city, arrival city, start time, end time, and duration, and given a source and destination, find the sequence of trains that gets you there at the earliest possible time.

MEDIUM | asked n/a |

Given a train schedule with departure city, arrival city, start time, end time, and duration, and given a source and destination, find the sequence of trains that gets you there at the earliest possible time.

```
def earliest_route(trains, source, target, layo
  ver=0):
```

34. Given a matrix with a rat, bread, cat, and obstacles, find a path from the rat to the bread that maximizes the minimum distance from cat.

MEDIUM | asked n/a |

Given a matrix with a rat, bread, cat, and obstacles, find a path from the rat to the bread that maximizes the minimum distance from cat.

```
def safest_value(grid):
```

35. Sort a nearly sorted (or K sorted) array.

MEDIUM | asked n/a |

Sort a nearly sorted (or K sorted) array.

```
def sort_k_sorted(arr, k):
```

36. Meeting Rooms III Variant

MEDIUM | asked n/a |

Given an array of meeting time intervals where each interval is [start, end], determine the minimum number of conference rooms required to hold all meetings without overlap. Variant: times are cyclic within a 24-hour period (meetings can wrap around midnight).

```
def min_rooms_sweep(meetings, day=DAY):
```

37. Playlist Shuffle No Adjacent Artists

MEDIUM | asked n/a |

Given a playlist of songs where each song has an artist, implement a shuffle function that minimizes (or eliminates) adjacent songs by the same artist.

```
def shuffle_playlist(songs, gap=2, strict=False
  , rng=random):
```

38. Find k pairs with smallest sums

EASY | asked 2025-11-01 |

You are given two **sorted** integer arrays arr1 and arr2 (each sorted in non-decreasing order), and an integer k. Consider all possible pairs formed by taking one element from arr1 and one element from arr2. For indices i and j, the pair is (arr1[i], arr2[j]) with sum arr1[i] + arr2[j]. Your task is to return the **k pairs** (arr1[i], arr2[j]) with the smallest sums. If there are fewer than k total possible pairs, return all possible pairs.

```
def k_smallest_pairs(a, b, k):
```